What Senior ML Engineers Actually Look For When Hiring

The No-BS Guide to Getting Hired in ML

From Someone Who Actually Hires ML Engineers

If you think knowing PyTorch and completing Andrew Ng's course makes you hireable, this will hurt. But it'll save you years of wasted effort.

January 9, 2026

Contents

Introduction

I've reviewed hundreds of ML resumes. I've hired for ML roles. And I've trained executives at companies that paid me 1,50,000 INR for an hour of training.

Key Insight

Here's what nobody tells you: We don't hire ML engineers. We hire software engineers who can think in ML.

This guide breaks down the **ACTUAL** hiring criteria — not the fluff you see on LinkedIn, but what makes someone go from "rejected in screening" to "let's make an offer."

1 The Reverse Engineering Framework

1.1 The Problem with Most Candidates

Most candidates show up with:

- Kaggle notebooks
- Course certificates
- GitHub repos with cat/dog classifiers
- "Experience with TensorFlow, PyTorch, Scikit-learn"

And they all get rejected. Why?

Key Insight

Because they're showing **OUTPUT**, not **PROCESS**. They're showing **TOOLS**, not **THINKING**.

When I evaluate someone, I'm reverse-engineering their decision-making process. Not what they built — but **HOW** they thought through constraints.

1.2 The Real Questions We Ask

1.2.1 Question 1: "Walk me through your most complex ML project"

Red Flag

Red Flag Answer:
"I built a sentiment analysis model with 94% accuracy using BERT..."
What this reveals: Tools and metrics, no context or thinking.

Green Flag

What We Want to Hear:
"I was solving X business problem. The constraint was Y. I initially thought Z approach would work, but discovered A limitation. So I pivoted to B, traded off C for D, and here's the production impact..."
What this reveals: Constraint navigation, trade-off thinking, business impact.

See the difference?

- First answer: Tools and metrics
- Second answer: Constraint navigation, trade-off thinking, business impact

1.2.2 Question 2: "Why didn't you use [obvious approach X]?"

This is a **trap question**. We KNOW approach X exists. We want to see if YOU know why it wouldn't work in YOUR context.

Red Flag

Red Flag: "Oh, I didn't think of that..."

Green Flag

What We Want:
"I considered X, but given [constraint 1] and [constraint 2], it would have resulted in [specific problem]. Instead, I chose Y because [production reasoning]."

This reveals:

- Do you think in constraints or just tutorials?
- Do you understand trade-offs or just copy-paste solutions?
- Can you defend technical decisions or just follow recipes?

2 The Stack-Based Evaluation

Here's my framework for evaluating candidates. I call it **Stack-Based Assessment** — and it's the same framework I use for my own learning.

2.1 Level 1: Core ML Understanding (Foundation)

Action Item

Not about: Memorizing algorithms
About: Can you explain WHY this algorithm for THIS problem?

2.1.1 Interview Question Example

"You have 3000 images, half are night-time cat photos, half are daytime dog photos, all unlabeled. Your model needs to work on both day/night, both species. You can't collect more data. Go."

What this reveals:

- Do they immediately jump to "fine-tune a pre-trained model"? (Tutorial thinking)
- Do they identify the core issue: confounded variables, class imbalance, distribution shift? (Production thinking)
- Do they think about data augmentation, semi-supervised learning, domain adaptation? (Constraint navigation)

2.2 Level 2: MLOps (The Filter)

Key Insight

This is where 80% of candidates fail.

We don't ask about MLOps directly. We ask:
"Your model is in production. 3 months later, accuracy drops from 85% to 62%. Walk me through your debugging process."

Red Flag

Red Flag Answer:
"I'd retrain the model with new data..."
Why this fails: Shows no understanding of production systems, monitoring, or root cause analysis.

Green Flag

What We Want:

1. "First, I'd check monitoring dashboards — is this data drift, concept drift, or label drift?"
2. "I'd look at feature distributions — are we getting out-of-distribution inputs?"
3. "I'd segment the errors — is accuracy dropping uniformly or for specific subgroups?"
4. "I'd check upstream data pipelines — has anything changed in data collection?"
5. "Only THEN would I consider retraining — and I'd set up A/B testing to validate the new model before full rollout."

Key Insight

This is production thinking. This is what separates engineers from tutorial followers.

2.3 Level 3: GenAI/Agents (Current Relevance)

Action Item

Not about: "I've used ChatGPT API"
About: Can you build SYSTEMS around LLMs?

2.3.1 Interview Question

"Build me a customer support agent that can handle refunds, product questions, and escalations. How do you approach this?"

Red Flag

Tutorial Answer:
"I'd use GPT-4 with a system prompt..."
Why this fails: Tool-focused, not system-focused. No consideration of constraints.

Green Flag

Production Answer:

- "First question: What's the latency requirement? Cost per query? Failure tolerance?"
- "I'd start with intent classification — separate routing layer before the LLM"
- "For refunds, deterministic logic ¿ LLM (cheaper, more reliable)"
- "For product questions, RAG with vector DB for context retrieval"
- "For escalations, confidence thresholding + human-in-the-loop"
- "Monitoring: track intent accuracy, response quality, escalation rate, user satisfaction"

See the difference? One is "use the shiny tool." The other is "build a SYSTEM with the right tool for each job."

3 The GitHub Deep Dive

Key Insight

Here's something most candidates don't expect: **I don't care about your star count. I care about your commit history.**

3.1 Red Flags

Red Flag

- 50 repos, all forked, zero original commits
- Repos with only Jupyter notebooks, no production code
- Projects that stop at 'model.py' — no deployment, no testing, no monitoring
- README that says 'achieved 94% accuracy' but no discussion of trade-offs
- Copy-pasted code with no customization

3.2 Green Flags

Green Flag

- **Consistent commit history** — shows sustained learning, not just pre-interview cramming
- **Evolution of a project** — I can see iterations, experiments, pivots
- **Production-ready code** — Docker files, CI/CD, tests, logging, error handling
- **Trade-off documentation** — 'Tried X, failed because Y, switched to Z'
- **End-to-end ownership** — Data pipeline → Training → Deployment → Monitoring

3.3 Example of What Gets My Attention

A repo that says:
"Built a recommendation system for [domain]. Initial approach: collaborative filtering. Problem: cold start for new users. Solution: hybrid model with content-based fallback. Deployed with FastAPI + Redis caching. Current latency: 45ms p95. Cost: $0.003 per 1000 requests."

This tells me:

- You understand the problem space
- You encountered real constraints
- You engineered around them
- You measure what matters (latency, cost, not just accuracy)

4 The Software Engineering Filter

Key Insight

Truth bomb: If you can't pass a mid-level software engineering interview, you won't get an ML role.
Because **60-70% of the job IS software engineering.**

4.1 What We Actually Test

4.1.1 1. Data Structures & Algorithms (Yes, Still Relevant)

Not LeetCode hard, but you should be able to:

- Implement a basic cache with TTL (relevant for model serving)
- Design a rate limiter (relevant for API endpoints)
- Handle streaming data processing (relevant for real-time inference)

Why? Because when your model is timing out in production, you need to know if it's the model, the data preprocessing, the I/O, or the network.

4.1.2 2. System Design (The Real Test)

"Design a real-time fraud detection system that processes 10,000 transactions per second."
What we're evaluating:

- Do they ask about latency requirements, false positive tolerance, data availability?
- Do they separate the hot path (fast rules + lightweight model) from cold path (complex model)?
- Do they think about feature store, model versioning, fallback strategies?
- Do they consider monitoring, alerting, model retraining pipelines?

Key Insight

This is where tutorial learners die. They can train a fraud detection model in a notebook. They cannot ENGINEER it into production.

4.1.3 3. Code Quality (Non-Negotiable)

I'll give you a take-home: *"Here's a messy data pipeline script. Refactor it."*
What I'm testing:

- Do you modularize it? (functions, classes)
- Do you add type hints?
- Do you add logging and error handling?

- Do you add tests?
- Do you add documentation?

Red Flag

If your submission comes back as "here's the same code but it runs faster" — **rejected**.

Green Flag

If it comes back as "here's modular, tested, documented, production-ready code with clear error messages and observability" — **interview scheduled**.

5 The Passion & Problem-Solving Test

This is the X-factor that separates "good candidate" from "hell yes, let's hire."

5.1 The Passion Question

"What's something in ML you've been obsessing over lately?"

Red Flag

Red Flag: "Uh... I've been doing Andrew Ng's course..."

Green Flag

What Gets Me Excited:

- "I've been experimenting with different quantization techniques for LLMs — trying to get 7B models running on consumer hardware without quality loss"
- "I've been reverse-engineering how ChatGPT does multi-turn context management — built a few experiments around conversation memory"
- "I've been exploring sparse attention mechanisms — how far can we push context windows while keeping compute reasonable?"

What this shows:

- You're learning beyond courses
- You're solving problems you CREATED for yourself
- You're thinking about production constraints (cost, speed, deployment)

5.2 The Problem-Solving Live Test

"Here's a business scenario. Build me a solution. You have 30 minutes. Go."

I don't care if the solution works perfectly. I care about:

- How you break down the problem
- What questions you ask
- What assumptions you state explicitly
- How you prioritize (MVP vs nice-to-have)
- How you think about validation

Green Flag

The Best Candidates:

- Spend 10 minutes asking questions and clarifying constraints
- Sketch out 2-3 approaches with trade-offs
- Pick one with clear reasoning
- Build something simple that WORKS
- Explain what they'd add with more time

Red Flag

The Worst Candidates:

- Jump straight to coding
- Try to build the perfect solution
- Get stuck debugging
- Have nothing working at the end

6 The Anti-Pattern Checklist

Here are the instant rejections. If you do these, we won't even proceed to technical interview.

6.1 Resume Red Flags

red!20 ✗ RED FLAG	✓ BETTER APPROACH
"Proficient in: Python, TensorFlow, PyTorch, Keras, Scikit-learn, Pandas, NumPy..." This is a library list, not skills. Everyone knows these.	"Built production ML pipelines serving 10M+ requests/day with ¡100ms p95 latency" Shows actual impact and constraints.
"Completed 15 ML courses from Coursera, Udemy..." Courses are learning, not experience.	"Self-taught ML engineer — built [specific project] solving [specific problem]" Shows initiative and results.
"Achieved 96% accuracy on [dataset]" Accuracy on toy datasets means nothing.	"Improved click-through rate by 23% using [approach] with [constraints]" Shows business impact and constraint navigation.

6.2 Interview Red Flags

✗ Red Flag

- **Cannot explain why they chose a specific approach**
  → They followed a tutorial, didn't make decisions
- **Only talks about model accuracy, never mentions latency/cost/reliability**
  → They think ML = model training, not ML = production system
- **Cannot debug a model failure scenario**
  → They've never dealt with production issues
- **Asks "what tools should I learn?"**
  → Tool-focused, not problem-focused

7 The Real Hiring Criteria

Let me be direct about what actually gets you hired:

7.1 Tier 1: Non-Negotiables (Must Have)

ALL FOUR REQUIRED

1. **Software Engineering Fundamentals** — You can build production-quality code
2. **Constraint Thinking** — You understand trade-offs: accuracy vs latency vs cost vs complexity
3. **End-to-End Ownership** — You can take a problem from data → model → deployment → monitoring
4. **Communication** — You can explain technical decisions to both engineers and business stakeholders

If you don't have these 4, doesn't matter what else you have.

7.2 Tier 2: Strong Differentiators (Nice to Have)

1. **Domain Expertise** — You deeply understand a specific vertical (fintech, healthcare, e-commerce)
2. **Production Battle Scars** — You've dealt with model drift, data quality issues, scalability problems
3. **Research Awareness** — You read papers, understand SOTA, can evaluate when to use new techniques
4. **System Design** — You can architect ML systems, not just models

7.3 Tier 3: The Multipliers (Rare)

1. **Teaching Ability** — You can explain complex ML concepts simply (scales your impact)
2. **Entrepreneurial Mindset** — You connect ML solutions to business outcomes
3. **Tool Building** — You've built internal tools, frameworks, or infrastructure

7.4 The Hiring Formula

- **Tier 1 alone:** Maybe an interview
- **Tier 1 + Tier 2 (2-3 items):** Strong candidate
- **Tier 1 + Tier 2 (all 4) + Tier 3 (1+):** Hell yes, competitive offer

8 The Action Plan

Okay, so you've realized most of your prep has been wrong. What now?

8.1 Month 1-2: Software Engineering Foundation

Action Item

Stop: Doing more ML courses
Start: Building production-quality code

- Refactor your existing projects with: tests, type hints, documentation, CI/CD
- Learn: Docker, Git workflows, API design, logging, error handling
- Build: A REST API that serves a simple ML model with proper error handling and monitoring

8.2 Month 3-4: Constraint-Based Learning

Action Item

Stop: Building projects on clean Kaggle datasets
Start: Solving problems with real constraints

- "Build a recommender system with ¡50ms latency requirement"
- "Build an image classifier that works on mobile devices (¡ 10MB model size)"
- "Build a chatbot with ¡ $0.01 per conversation cost"

Document everything: What constraint? What trade-off? What solution? What impact?

8.3 Month 5-6: Production MLOps

Action Item

Stop: Training models in notebooks and calling it done
Start: Full lifecycle projects

- Deploy a model to production (even if it's just Hugging Face Spaces)
- Set up monitoring (track latency, error rates, data drift)
- Build a retraining pipeline
- Implement A/B testing for model comparison

8.4 Continuous: Stack-Based Depth

Core ML: Pick 3-5 algorithms, understand them DEEPLY — not just how to use them, but when/why/trade-offs
MLOps: Build your own mini-MLOps stack (even if local) — versioning, monitoring, deployment
GenAI/Agents: Build something BEYOND "ChatGPT wrapper" — RAG, tool use, multi-agent systems
System Design: Study ML system design — fraud detection, recommendation, search, ranking

Closing: The Path Forward

Look, I'm not saying this to discourage you. I'm saying this because the industry is full of people selling you shortcuts that don't exist.

Key Insight

The good news? If you actually do what I just outlined, you'll be in the top 5% of candidates. Because 95% won't.

They'll keep doing courses. Keep building toy projects. Keep wondering why they're not getting interviews.

The Path is Clear

1. Build software engineering skills FIRST
2. Learn to think in constraints and trade-offs
3. Build end-to-end production projects
4. Document your decision-making process
5. Show up to interviews ready to THINK, not just recite

That's what senior ML engineers look for.
That's what gets you hired.

Because I'm not here to sell you a dream. I'm here to show you the system.